Advanced Programming (OOPS) (PCCCS495)
Smart Parking Management System

Name - Srinjoy Aich
Department - CSE(IoT)
Enrollment No.- 12024052019007
Roll No.-11
Instructor Name - Dr. Soumadip Biswas & Dr. Swagatm Basu
Semester - Spring 2026

1. Problem Statement

In busy urban areas, finding a parking space is often difficult and time-consuming. Drivers frequently spend unnecessary time searching for available parking spots, which leads to traffic congestion and inefficient parking management. Traditional parking systems rely on manual processes that are prone to errors and inefficiencies.

The Smart Parking Management System is designed to automate the process of parking allocation. It efficiently manages parking slots, records vehicle entry and exit times, and calculates parking fees based on duration and vehicle type. The system ensures optimal utilization of parking space and reduces human intervention.

2. System Design

UML Class Diagram

```
            Vehicle (Abstract)
            ------------------
            - number : String
            - type : String
            + calculateFee()

              /      |       \
           Car     Bike     Truck

ParkingSlot -------- Vehicle
      |
      v
    Ticket -------- Vehicle

ParkingLot
    |
    v
Main (Controller)
```

Description of Major Classes

- **Vehicle (Abstract Class):**
  Base class containing common properties like vehicle number and type.
- **Car, Bike, Truck:**
  Derived classes implementing specific fee calculation.
- **ParkingSlot:**
  Represents a parking slot and manages its occupancy.
- **Ticket:**
  Stores entry time, exit time, and calculates total parking fee.
- **ParkingLot:**
  Core class managing parking operations such as entry, exit, and slot allocation.
- **Main Class:**
  Provides a menu-driven interface for user interaction.

Package Structure

```
src/
  ├── Vehicle.java
  ├── Car.java
  ├── Bike.java
  ├── Truck.java
  ├── ParkingSlot.java
  ├── Ticket.java
  ├── ParkingLot.java
  └── Main.java
```

Interaction Flow

1. User selects parking option
2. System assigns available slot
3. Ticket is generated with entry time
4. On exit, system calculates fee
5. Receipt is displayed

3. OOP Concepts Demonstrated

(i)Encapsulation

Where Used: Vehicle, ParkingSlot
Why: Protects data and ensures controlled access

Code Snippet:

```
private String number;


public String getNumber() {

   return number;

}
```

Explanation:
Data is hidden using private variables and accessed via getter methods.

(ii)Inheritance

Where Used: Car, Bike, Truck → Vehicle

Code Snippet:

```
class Car extends Vehicle {
  public Car(String number) {
      super(number, "Car");
  }
}
```

Explanation:
Reduces redundancy and allows reuse of base class properties.

(iii)Polymorphism

Where Used: calculateFee() method

Code Snippet:

```
@Override
public double calculateFee(long hours) {
  return hours * 20;
}
```

Explanation:
Different classes implement the same method differently.

(iv)Abstraction

Where Used: Vehicle class

Code Snippet:

```
abstract class Vehicle {
  public abstract double calculateFee(long hours);
}
```

Explanation:
Hides implementation details and provides a blueprint.

(v)Classes & Objects

Where Used: Entire system

Explanation:
Real-world entities are represented as classes and objects.

4. Implementation Highlights

Snippet 1: Parking Allocation

```
for (ParkingSlot slot : slots) {
  if (!slot.isOccupied()) {
      slot.parkVehicle(vehicle);
      break;
  }
```

```
}
```

Justification:
Ensures automatic allocation of first available slot.

Snippet 2: Fee Calculation

```
long hours = Duration.between(entryTime, exitTime).toHours();

if (hours == 0) hours = 1;

return vehicle.calculateFee(hours);
```

Justification:
Accurate billing based on duration.

Snippet 3: Ticket Generation

```
System.out.println("Vehicle: " + vehicle.getNumber());

System.out.println("Fee: ₹" + bill);
```

Justification:
Provides user-friendly receipt output.

5. Testing & Error Handling

(i)Test Cases

- Valid vehicle entry and exit
- Multiple vehicle types
- Full parking condition
- Invalid vehicle number

(ii)Edge Cases

- Parking when full
- Exit without entry
- Minimum parking time (1 hour)

(iii)Failure Scenarios

- Invalid input handled using condition checks
- Missing vehicle handled using null checks

6. Git Workflow

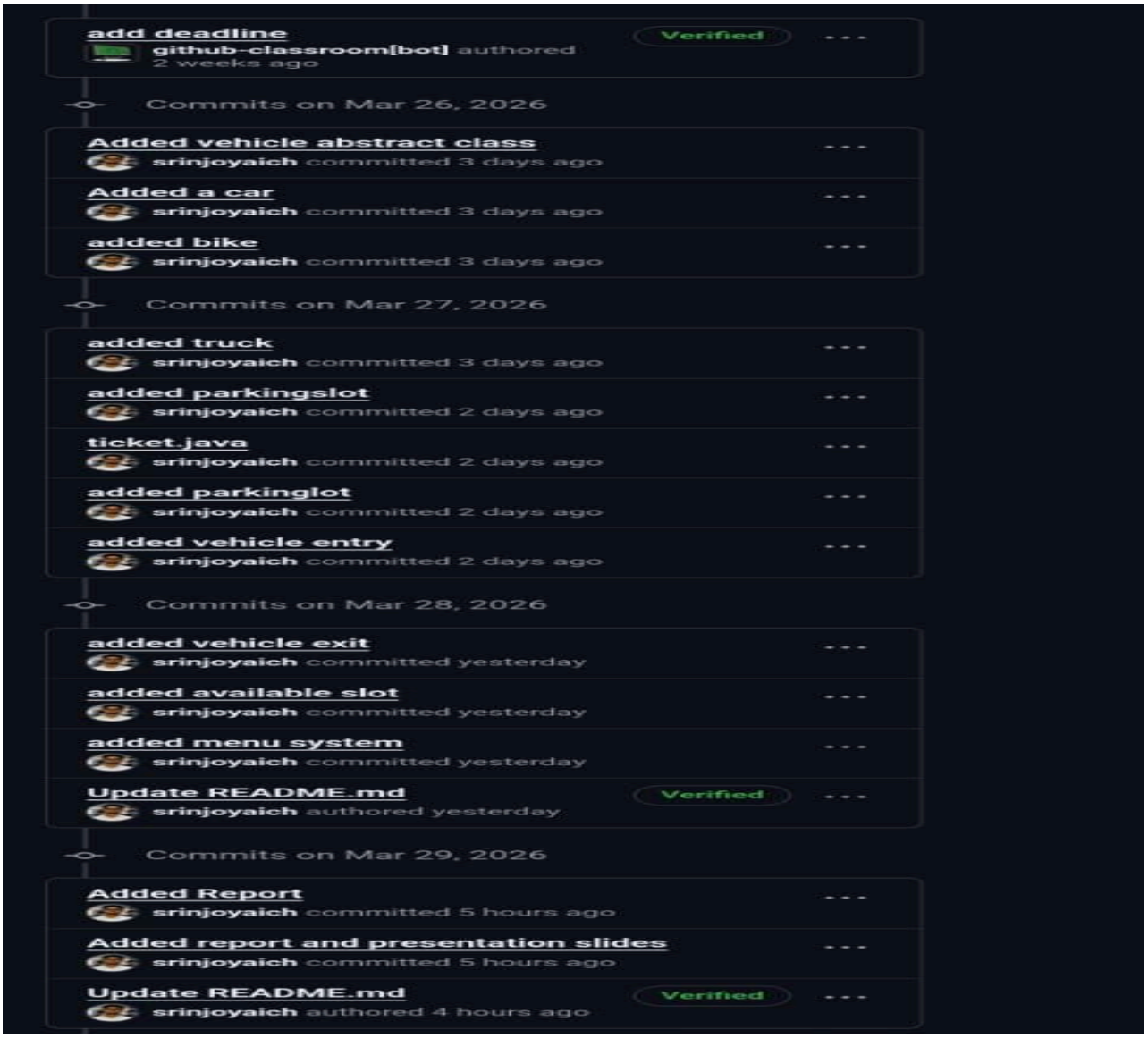
Development Process

- Project developed using incremental commits
- Each feature implemented step-by-step
- Minimum 10 meaningful commits maintained

Example Progression

1. Vehicle class
2. Subclasses (Car, Bike, Truck)
3. ParkingSlot
4. Ticket system
5. ParkingLot logic
6. Main interface

Screenshot



7. Conclusion & Future Scope

Conclusion

The Smart Parking Management System successfully demonstrates the application of object-oriented programming concepts to build an efficient and scalable solution. It automates parking allocation, reduces manual effort, and ensures accurate fee calculation.

Future Scope

- GUI implementation using Java Swing/JavaFX
- Database integration for persistent storage
- Mobile app integration
- Real-time parking tracking using IoT

(P.T.O.)

Attachments : -

1. Copy of Initial Project Proposal (Attached)
2. Copy of review mail received (with APPROVED_MINOR)